Relationship between Pointers and 1D, 2D and 3D Arrays in C

06 November 2026 16:52

Pointers and arrays are two of the most important concepts in C programming. They are closely related because an **array name represents the address to its first element in most expressions and pointers can be used to access array elements.**

However, the relationship becomes slightly different when we work with 1D2D and 3D arrays.

What is an Array?
An array is a collection of elements of the same data type stored in contiguous memory location.

```c
#include<stdio.h>
int main(int argc, char const *argv[])
{
    int a[5] = {10, 20, 30, 40, 50};
    printf("%d", a[0]);
    return 0;
}
```

Here:

- a is an integer array
- It contains five elements.
- Each element occupies sizeof(int) bytes.
- Elements are stored in consecutive memory locations.

What is a pointer?
A pointer is a variable that stores the memory address of another variable.

Example:

```c
#include<stdio.h>
int main(int argc, char const *argv[])
{
    int x = 100;
    int *p = &x;
    printf("Value of x = %d\n", x);
    printf("Address of x = %p\n", &x);
    printf("Value of p = %p\n", p);
    printf("Value pointed by p = %d\n", *p);
    return 0;
}
```

Output:

```
Value of x = 100
Address of x = 00000040a51ff8d4
Value of p = 00000040a51ff8d4
Value pointed by p = 100
```

Understand these operators

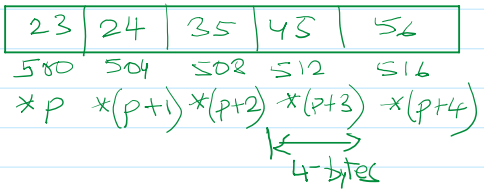
Operator	Meaning
&	Address-of operator
*	Dereference operator
p	Stores the address
*p	Value available at that address

```c
#include<stdio.h>
int main(int argc, char const *argv[])
{
    int a[5] = {10, 20, 30, 40, 50};
    int *p = a; // equivalent to int *p = &a[0];
    printf("%d", a[0]);
    return 0;
}
```

Why?
Because in this expression, the array name a decays into a pointer to the first element

int a[5] = { 23, 24, 35, 45, 56};

int *p = a;




```c
#include<stdio.h>
int main(int argc, char const *argv[])
{
    int a[5] = {10, 20, 30, 40, 50};
    int *p = a; // equivalent to int *p = &a[0];
    printf("a[]\t*p\n");
    printf("%d\t", a[0]);
    printf("%d\n", *p);

    printf("%d\t", a[1]);
    printf("%d\n", *(p + 1));
    printf("%d\t", a[2]);
    printf("%d\n", *(p+2));
    printf("%d\t", a[3]);
    printf("%d\n", *(p+3));

    printf("%d\t", a[4]);
    printf("%d\n", *(p+4));
```

```
    return 0;
}
```

Output:

a[]	*p
10	10
20	20
30	30
40	40
50	50

Both approaches produce the same values.

Important formula:

a[i] = *(a+i)

Similarly:

p[i] = *(p+i)

This relationship is the foundation of pointer - based array programming.

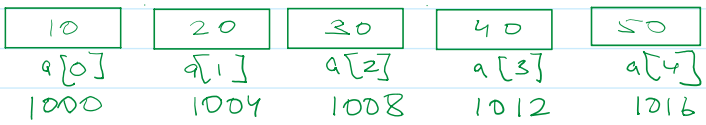
Relationship between pointers and 1D Arrays

A 1-dimensional array contains elements arranged in a single sequence.

```
int a[5] = {10, 20, 30, 40, 50}
int *p = a;
```

Memory representation (assuming a four - byte integer):

1D array memory layout



p points to 1st element of an array

*p gives the value of a[0]

Array Notation

a[0]
a[1]
a[2]
a[3]
a[4]

Pointer notation

*(p+0) / *p
*(p+1)
*(p+2)
*(p+3)
*(p+4)

Notice that when we write p+1, the pointer moves by the size of one integer, not by 1 byte.

If an integer occupies 4 bytes

p+1 = p+ 1*sizeof(int) ← data-type.

```c
#include<stdio.h>
#define SIZE 5
int main(int argc, char const *argv[])
{
    int a[SIZE] = {10, 20, 30, 40, 50};
    int *p = a; // equivalent to int *p = &a[0];
    printf("a[]\t*p\n");
    int i;
    for(i = 0; i < SIZE; i++){
        printf("%d\t%d\n", a[i], *(p + i));
    }
    return 0;
}
```

Output:

```
a[]     *p
10      10
20      20
30      30
40      40
50      50
```

```c
#include<stdio.h>
#define SIZE 5
int main(int argc, char const *argv[])
{
    int a[SIZE] = {10, 20, 30, 40, 50};
    int *p = a; // equivalent to int *p = &a[0];
    printf("a[]\t*p\n");
    int i;
    for(i = 0; i < SIZE; i++,p++){
        printf("%d\t%d\n", a[i], *(p));
    }
    return 0;
}
```

here the content of pointer is updated by 4-bytes.

Key concept: A pointer to an integer can traverse a 1D integer array because each pointer increment moves to the next integer.

Relationship between pointers and 2D arrays

A two - dimensional array is an array of arrays.
It is commonly used to represent matrices, tables and grids.

```
int a[3][4] ={
        {10, 20, 30, 40},
        {50, 60, 70, 80},
        {90, 100, 110, 120}
     };
```

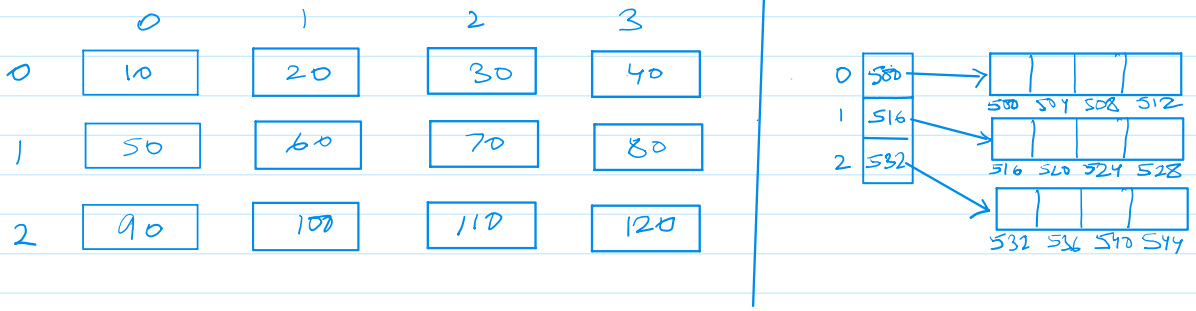
This array contains:
- Three rows.
- 4 columns.
- Total 12 integer elements.

Memory representation of a 2D array.

In C a 2D-array is stored in **row major order.**

That means all elements of the first row are stored first, followed by the 2nd row and then the 3rd row.

2D array: a[3][4]:




```
int a[3][4] ={
        {10, 20, 30, 40},
        {50, 60, 70, 80},
        {90, 100, 110, 120}
     };
```

What is the meaning of a?

The array name 'a' decays into a pointer to its first row.

The first row is a[0],

Its type is a[4]

Therefore, the pointer type is:
int (*)[4]

This is called a pointer to an array of 4 integers.

The correct declaration is:

int (*p)[4] = a;

int (*p)[4] = a;

Pointer to an integer

Pointer to an array of 4 integers

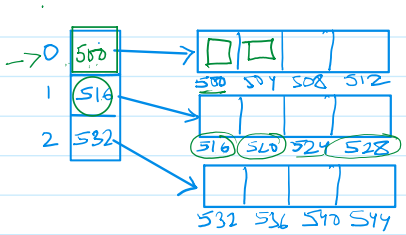
int *p[4]; ← Array of 4 integer pointer

Note the parentheses.

(*p)[4] and int *p[4]; are completely different

```
int a[3][4] ={
        {10, 20, 30, 40},
        {50, 60, 70, 80},
        {90, 100, 110, 120}
     };

 int (*p)[4] = a;
```



*(p+0) = 500

fixed

((p+0)+0)

*(500+0)

((p+0)+1)

*(500 + 1x sizeof(int))

*(504)

We can access elements using:

Array notation	Pointer notation
a[0][0]	*(*(p+0)+0)
a[0][1]	*(*(p+0)+1)
a[0][2]	*(*(p+0)+2)
a[0][3]	*(*(p+0)+3)
a[1][0]	*(*(p+1)+0)
…	…

The general formula is:

a[i][j] = *(*(p+i)+j)

Breakdown:

i. **p + i** moves to row i.
ii. ***(p+i)** Represents row *i* and decays into a pointer to the first element.
iii. ***(p+i)+j** Moves to column *j*.
iv. The outer * retrieves the actual element.

```c
#include<stdio.h>
int main(int argc, char const *argv[])
{
    int a[3][4] ={
            {10, 20, 30, 40},
            {50, 60, 70, 80},
            {90, 100, 110, 120}
        };
```

```
        int (*p)[4] = a;
        printf("2d array elements:\n");
        int i, j;
        for(i = 0; i < 3; i++, printf("\n")){
            for(j = 0; j < 4; j++){
                printf("%d\t", *(*(p+i)+j));
            }
        }
    return 0;
}
```

Output:

```
2d array elements:
10      20      30      40
50      60      70      80
90      100     110     120
```

```
#include<stdio.h>
int main(int argc, char const *argv[])
{
    int a[3][4] ={
            {10, 20, 30, 40},
            {50, 60, 70, 80},
            {90, 100, 110, 120}
        };

        int (*p)[4] = a;
        printf("2d array elements:\n");
        int i, j;
        for(i = 0; i < 3; i++, printf("\n")){
            for(j = 0; j < 4; j++){
                // printf("%d\t", *(*(p+i)+j));
                printf("%d\t", p[i][j]);
            }
        }
    return 0;
}
```

Alternative pointer relationship

((p+i)+j)

p points to an array of 4 integers, *p + 1* moves forward by an entire row

If sizeof(int) = 4:

p + 1 = p + 16 bytes

This is an important difference from a simple int * pointer.